

The Backend: APIs & Services

2026-04-29 · lively date Hase

MAKING YOUR CODE SPEAK TO THE WORLD OVER HTTP.

The Why

Block 4 left us with `app/classifier.py`, a function that turns a 28-by-28 array of pixels into a prediction dictionary. The function works. `predict.py` proves it on real MNIST samples. What it cannot do is be reached from anywhere except the same Python process that imported it. A frontend in a browser, a teammate on another machine, an automated test runner, none of them can call `classify_batch` as it stands.

ACROSS TEN LECTURE BLOCKS WE BUILD `PIXELWISE`. This block is the first that exposes the application to the outside world. Every client that follows, the browser frontend in Block 7, the CI runner in Block 8, the monitoring `curl` in Block 9, talks to the service through the contract you design here.

HTTP IS THE PROTOCOL that every browser, every CLI tool, and every web service in this course speaks. A client opens a TCP connection, sends a text-shaped request, and reads a text-shaped response back. The request says which method, which path, which headers, and which body; the response says which status, which headers, and which body. That is the entire protocol surface most applications need.

A SERVICE is what `predict.py` becomes when you wrap it in HTTP and run it under a supervisor. `uvicorn` is the HTTP server that hosts the FastAPI application; `systemd` is the supervisor that keeps `uvicorn` running, restarts it when it crashes, and captures its logs. Together they turn a Python script into something that survives a reboot, a crash, or a closed terminal window.

`PIXELWISE` now has a working classifier behind a Python function. A function call is the wrong interface for everything that is not Python running on the same machine. This block moves the function behind an HTTP endpoint so any client that can speak HTTP can reach it.

REST is the architectural style that maps the HTTP verbs onto application resources. Each resource has a stable URL; each verb has a clear meaning. We use REST because it is the convention every frontend, every tool, and every framework in this stack expects, not because it is the only valid choice.

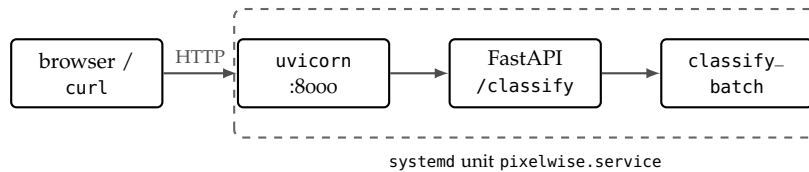


Figure 1: Request flow for a single classification. The client sends an HTTP request to uvicorn; FastAPI dispatches it to the `/classify` handler, which calls `classify_batch` from Block 4. The whole stack runs under a `systemd` unit that restarts it if it crashes.

Hands On Experience

CONSIDER THIS SCENARIO: a teammate cloned PixelWise yesterday and got the smoke test working. This morning they want to call your classifier from a small JavaScript demo in their browser. The browser cannot import a Python function. Your teammate cannot SSH into your machine. Without an HTTP layer, the work you did in Block 4 is unreachable to anyone who is not sitting at your laptop.

THE CONTRACT BECOMES THE PRODUCT. Once the service is running, the request and response shapes are what every other component in the system depends on. Change them silently and the frontend breaks; document them clearly and new clients can integrate without ever reading your Python. Pydantic is the tool that lets us write the contract in the same place as the code that enforces it.

THIS FIFTH BLOCK introduces the backend service layer for PixelWise. By the end you will have

- understood HTTP and REST well enough to design and probe an endpoint,
- written request and response contracts as Pydantic models,
- wrapped `classify_batch` in a FastAPI application,
- added API key authentication and basic rate limiting,
- run the service under `systemd`, observed crashes, and read its logs with `journalctl`.

THE SCRIPT-TO-SERVICE JUMP. A script runs once, prints, exits. A service runs continuously, accepts requests, returns responses, survives crashes. The work in this block is what turns the first into the second; the classifier itself does not change.

WHY `systemd`? Running `uvicorn` in a terminal works until you close the terminal. Running it under `systemd` survives reboots, restarts on crash, and writes its output to a structured log you can query later. The classroom requirement and the production requirement are the same.

HTTP and REST

EVERY WEB INTERACTION IS A REQUEST AND A RESPONSE. The client sends a request that names a method, a path, and an optional body; the server sends back a status code, headers, and an optional body. The protocol carries no state of its own: each request is interpretable on its own, without knowledge of any previous request from the same client.

STATUS CODES are three-digit numbers grouped by leading digit. 2xx means success, 4xx means the client made a mistake, 5xx means the server made a mistake. The codes you see most: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 422 Unprocessable Entity, 429 Too Many Requests, 500 Internal Server Error.

REST PRINCIPLES layer convention on top of the protocol. Every distinct thing the API exposes is a resource with a stable URL: `/classify`, `/results`, `/health`. Methods do what their names suggest. Responses are JSON. Two clients written by two teams, six months apart, agree on what the API does without ever speaking, because the conventions remove ambiguity.

THE INTERACTIVE DEFINITION OF AN API is the documentation that ships with the running service. FastAPI generates this automatically from your code, at `/docs`, and exposing it changes the social contract: a teammate does not need to read your source to integrate. Block 3's `DEBUG=true` flag controls whether `/docs` is exposed, so the same service is self-documenting in development and silent in production.

METHODS YOU WILL USE. GET retrieves; POST submits; PUT replaces; DELETE removes. PixelWise uses POST for `/classify` (the client submits a drawing) and GET for `/results` and `/health` (the client reads state).

STATELESS is the load-bearing property of HTTP. The server keeps no per-client memory between requests; everything the server needs to act is in the current request, including any authentication token. This is what lets you scale from one server to many: any worker can handle any request.

HTTP status codes reference:
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>
 REST overview:
<https://restfulapi.net/>

Exercise: HTTP, REST, and curl

CATCHING UP VIA THE REFERENCE TAG. The course maintains a public reference repository at <https://github.com/schutera/pixelwise> with a tag at the end of every block. v0.3 marks the end of Block 4; v0.4 will mark the end of this block. If your repository has drifted, reset to v0.3 before starting:

```
git fetch origin
git reset --hard v0.3
```

PROBE A PUBLIC API before writing your own. HTTP is best learnt against something you did not build, so you can read status codes and headers without already knowing what they should be. On the dev VM, with the virtual environment active:

```
curl -i https://httpbin.org/get
curl -i -X POST https://httpbin.org/post \
  -H "Content-Type: application/json" \
  -d '{"hello": "world"}'
```

NOW NAME THE VERBS. For each of the three endpoints PixelWise will expose, POST /classify, GET /results, GET /health, write a one-line justification: which method does it use, why, and what status code does success look like? The naming exercise pays off when you read other people's APIs and discover most of them follow the same conventions.

EXERCISES are where the theory above turns into keystrokes. Each theory section is followed by its exercise, so you can read a concept and practise it immediately. At the end of the session, take a snapshot named B5-complete, your safety net for the next block.

READ EACH SECTION OF THE RESPONSE. HTTP/2 200 is the status line. The lines that follow are headers; the blank line marks the boundary; the JSON below it is the body. Every HTTP response in this course has the same shape; once it is familiar, nothing about the protocol surprises you.

THE RIGHT VERB for inference is POST because the request carries a body and the server creates a record. GET carries no body in well-behaved clients and caches at every layer in between, neither of which fits a per-image classification.

API Contracts with Pydantic

AN API CONTRACT DEFINES what the client sends and what the server returns, precisely enough that a misunderstanding becomes an error rather than a silent bug. FastAPI uses Pydantic models to make contracts executable: the server validates every request against the model and rejects anything that does not match.

THE REQUEST AND RESPONSE MODELS for POST /classify:

```
from pydantic import BaseModel

class ClassifyRequest(BaseModel):
    pixels: list[list[int]] # 28x28, values 0-255

class ClassifyResponse(BaseModel):
    prediction: str # element of CLASSES
    confidence: float # probability of top class
    scores: dict[str, float] # probability per class
```

INPUT VALIDATION IS THE FIRST LINE OF DEFENCE. What happens if the client sends a flat list instead of a 28-by-28 grid? A list whose inner length is 27? Negative integers? A field that is not even named pixels? Without a contract, every one of these reaches your code as untyped Python and has to be checked manually. With Pydantic, every one of them is rejected at the boundary with a structured error:

```
{
  "detail": [
    {
      "loc": ["body", "pixels"],
      "msg": "value is not a valid list",
      "type": "type_error.list"
    }
  ]
}
```

THE CONTRACT IS THE DOCUMENTATION. /docs reads the Pydantic models and renders them as an interactive OpenAPI page where any visitor can issue a request and see the response. There is no second source of truth that drifts out of sync; if the model changes, the documentation changes with it.

BIDIRECTIONAL CONTRACT. The response model documents what the client receives, with the same precision as the request model documents what the client sends. The Block 7 frontend consumes ClassifyResponse; it knows the keys, the types, and the shape of scores before any HTTP request leaves the browser. Pydantic: <https://docs.pydantic.dev/>

HTTP 422 Unprocessable Entity is the standard response for a request that parses as JSON but fails schema validation. The client knows the request was structurally wrong; the server knows it never reached the handler. 500 would be a server bug; 422 is a client error caught at the boundary.

Exercise: The Pydantic Contract

TURN THE CONTRACT ABOVE INTO CODE. Inside the PixelWise repository on dev, install FastAPI, Pydantic, and uvicorn into the virtual environment, then update the pinned requirements:

```
source .venv/bin/activate
pip install fastapi uvicorn[standard] \
    slowapi pydantic
pip freeze > requirements.txt
```

WRITE THE REQUEST AND RESPONSE MODELS in app/main.py:

```
from pydantic import BaseModel

class ClassifyRequest(BaseModel):
    pixels: list[list[int]]

class ClassifyResponse(BaseModel):
    prediction: str
    confidence: float
    scores: dict[str, float]
```

TRY A TINY SCRIPT that exercises the model directly, without any HTTP server involved:

```
from app.main import ClassifyResponse
r = ClassifyResponse(prediction="7",
    confidence=0.94, scores={"7": 0.94})
print(r.model_dump_json())
```

The model serialises to JSON with the same keys the API will return. This is the single source of truth for the response shape from this exercise onwards.

THE SHAPE LIVES IN TWO PLACES
NOW: `ClassifyResponse` in Python and the dictionary `classify_batch` returns from Block 4. The two should match exactly. A field named `prediction` in one and `predicted` in the other is exactly the kind of mismatch the contract is supposed to prevent; keeping them aligned is part of the work.

FastAPI in Practice

FASTAPI IS A MODERN PYTHON WEB FRAMEWORK built on top of Pydantic and the ASGI standard. It generates interactive API documentation automatically, validates inputs and outputs against the models you declared, and runs on uvicorn, an asynchronous HTTP server fast enough that your bottleneck will be the model, not the framework.

A PATH OPERATION is a function decorated with an HTTP method and a URL path. The decorator binds the function to the route; the type hints on its parameters become the validation contract.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
def health():
    return {"status": "ok", "model_version": "v1"}

@app.post("/classify",
         response_model=ClassifyResponse)
def classify(req: ClassifyRequest):
    arr = np.array(req.pixels,
                  dtype=np.uint8)[np.newaxis]
    return classify_batch(arr)[0]
```

curl IS YOUR FIRST-CLASS DEBUGGING TOOL. A handler that works from /docs but fails from curl is nearly always a missing header. A handler that fails from both is a code bug. Reach for curl the moment something is unclear:

```
curl http://localhost:8000/health
curl -X POST http://localhost:8000/classify \
  -H "Content-Type: application/json" \
  -H "X-API-Key: your-secret-key" \
  -d '{"pixels": [[0,0,...], [0,255,...], ...]}'
```

THREE ENDPOINTS cover the surface this block exposes: POST /classify performs inference, GET /results is a stub that Block 6 fills in once persistence exists, and GET /health reports liveness so a monitor can tell whether the service is up.

FastAPI: <https://fastapi.tiangolo.com/>
 Pydantic: <https://docs.pydantic.dev/>
 uvicorn: <https://www.uvicorn.org/>
 ASGI: <https://asgi.readthedocs.io/>

THE DECORATOR IS THE CONTRACT. @app.post("/classify", response_model=...) tells FastAPI which method and path the function serves, and which model the response must conform to. A return value that does not match ClassifyResponse is rejected by the framework, not by a future bug report.

curl: <https://curl.se/>
 -X sets the HTTP method.
 -H sets a header.
 -d sets the request body.
 -i prints the response headers alongside the body, useful when a status code surprises you.

Exercise: Building app/main.py

WIRE THE PATH OPERATIONS FROM THE THEORY ABOVE INTO A WORKING SERVICE. Extend app/main.py with three endpoints:

```
from fastapi import FastAPI
import numpy as np
from app.classifier import classify_batch

app = FastAPI()

@app.get("/health")
def health():
    return {"status": "ok", "model_version": "v1"}

@app.get("/results")
def results():
    return {"results": [],
            "note": "persistence added in Block 6"}

@app.post("/classify",
          response_model=ClassifyResponse)
def classify(req: ClassifyRequest):
    arr = np.array(req.pixels,
                   dtype=np.uint8)[np.newaxis]
    return classify_batch(arr)[0]
```

TEST WITH CURL FROM ANOTHER TERMINAL:

```
curl http://localhost:8000/health
curl -X POST http://localhost:8000/classify \
  -H "Content-Type: application/json" \
  -d '{"pixels": [[0,0,...], ...]}'
```

A 28-by-28 zero array returns a low-confidence prediction; a real digit loaded from MNIST returns the right answer. The first end-to-end loop is when this works.

RUN IT LOCALLY: uvicorn
 app.main:app -reload -port 8000.
 The -reload flag restarts on every code change, which is what you want in development and never want in production. Open <http://localhost:8000/docs> to see the auto-generated OpenAPI page; click “Try it out” on POST /classify to issue a real request without leaving the browser.

SEND A DELIBERATELY WRONG SHAPE.
 Submit a 27-by-27 grid and read the 422 response. Pydantic rejects the request before the handler runs, with a structured error that tells the client exactly which field was wrong. This is the input validation boundary doing its job.

Authentication and Rate Limiting

EVEN A CLASSROOM SERVICE NEEDS BOTH. 30 students hitting the same endpoint in the same minute is enough load to surface a rate-limit problem on the first day. A leaking debug build is enough to see the API key reach a place it does not belong. Authentication and rate limiting are layered defences: the first answers “who is calling”, the second answers “how often”.

API KEY AUTHENTICATION is the simplest form that fits this course. The client sends a secret in a header; the server compares it to the value in .env and rejects requests that do not match.

```
from fastapi import Header, HTTPException

def verify_api_key(x_api_key: str = Header(...)):
    if x_api_key != os.getenv("SECRET_API_KEY"):
        raise HTTPException(
            status_code=401,
            detail="Invalid API key")
```

RATE LIMITING prevents both abuse and accident. slowapi attaches a per-route decorator that counts requests per client IP and rejects them with HTTP 429 when the limit is exceeded.

```
from slowapi import Limiter
from slowapi.util import get_remote_address

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter

@app.post("/classify")
@limiter.limit("30/minute")
def classify(req: ClassifyRequest):
    ...
```

PYDANTIC, THE API KEY MIDDLEWARE, AND THE RATE LIMITER together form a three-stage filter at the boundary of the application. A request that arrives must parse as JSON, match the schema, present a valid key, and stay within the rate budget before any inference code runs. Each stage rejects with a different status code, so a client failure tells you exactly which contract was violated.

WHY A HEADER? Headers travel separately from the body and the URL, so the secret never ends up in the path component of an access log. Putting the key in a query parameter would write it into every log line the request touched on the way to the server.

WHERE DOES RATE LIMITING BELONG?

Application-level limits are easy to write and travel with the code. Reverse-proxy limits, which we add in Block 7 with Nginx, run in front of the application and protect it even when it is overwhelmed. Both layers are useful; the latter is harder to bypass. slowapi: <https://github.com/laurentS/slowapi>

Exercise: API Key Auth and Rate Limiting

ADD THE DEFENSIVE LAYER FROM THE THEORY ABOVE TO THE SERVICE. Add API key middleware to `app/main.py`:

```
from fastapi import Header, HTTPException, Depends
import os

def verify_api_key(x_api_key: str = Header(...)):
    if x_api_key != os.getenv("SECRET_API_KEY"):
        raise HTTPException(
            status_code=401, detail="Invalid API key")

@app.post("/classify",
         response_model=ClassifyResponse,
         dependencies=[Depends(verify_api_key)])
def classify(req: ClassifyRequest):
    ...
```

ADD RATE LIMITING with `slowapi`:

```
from slowapi import Limiter
from slowapi.util import get_remote_address
from slowapi.middleware import SlowAPIMiddleware

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_middleware(SlowAPIMiddleware)

@app.post("/classify")
@limiter.limit("30/minute")
def classify(req: ClassifyRequest):
    ...
```

TEST ALL THREE FAILURE MODES:

- Missing header: 422.
- Wrong key: 401.
- More than 30 requests in a minute: 429.

Each one should reach curl as a status code rather than as a crash; that is the entire point of the boundary.

THE HEADER TRAVELS IN EVERY REQUEST. `X-API-Key` is a custom header name; FastAPI's `Header(...)` maps it to the function parameter automatically. A missing header returns 422 (Pydantic-level), a wrong value returns 401 (your handler).

WATCH A 429 HAPPEN. A small Bash loop firing 40 curl requests at the endpoint inside the same minute should see the last ten return 429 Too Many Requests. The remaining requests stay rejected until the window rolls over.

From Dev to Production: systemd

uvicorn RUNS FINE FOR TESTING, but one Ctrl+C and it is gone. A closed terminal kills it. A reboot kills it. A panic in a worker kills it without restart. None of that is the behaviour of a service.

systemd is the OS-level supervisor that comes with Ubuntu. It starts services on boot, restarts them when they crash, captures their output to a structured log, and exposes a uniform interface for operators. A systemd unit file describes the service well enough that the operating system can run it without you in the room:

```
# /etc/systemd/system/pixelwise.service
[Unit]
Description=PixelWise API
After=network.target

[Service]
User=produser
WorkingDirectory=/opt/pixelwise
EnvironmentFile=/opt/pixelwise/.env
ExecStart=/opt/pixelwise/.venv/bin/uvicorn \
    app.main:app --host 0.0.0.0 --port 8000
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
```

systemctl is the operator's interface to systemd. start, stop, and restart are the verbs you reach for during deployment; status reports the current state and the last few lines of output; enable marks the service as one that should start on boot.

```
sudo systemctl start pixelwise
sudo systemctl status pixelwise
sudo systemctl restart pixelwise
sudo systemctl enable pixelwise
```

IN PRODUCTION you would run gunicorn in front of uvicorn to spawn multiple worker processes, giving you concurrency and crash isolation. For our classroom setup, a single uvicorn worker behind systemd is sufficient. gunicorn is standard in production Python

WE USE A FRACTION of what systemd can do. It also manages timers (cron with dependency awareness), socket activation (start on connection), resource limits (cap CPU/memory per service), and dependency ordering (start the database before the app). It is the init system of virtually every modern Linux server.

systemd: <https://www.freedesktop.org/wiki/Software/systemd/>

journalctl is where you go first when something breaks. journalctl -u pixelwise prints every log line from your service since boot; -f follows new lines as they arrive, the way tail -f follows a file; -since "5 min ago" narrows the window when the bug is recent.

deployments; students who deploy beyond this course will encounter it within the first week.

Exercise: Running as a systemd Service

PROMOTE THE SERVICE FROM A FOREGROUND uvicorn TO A SUPERVISED systemd UNIT. Push your code to GitHub from dev, then SSH into the server and pull:

```
# on dev
git add app/main.py requirements.txt .env.example
git commit -m "Add FastAPI service with auth and limits"
git push
```

```
# on prod
ssh produser@192.168.56.11
cd /opt/pixelwise
git pull
source .venv/bin/activate
pip install -r requirements.txt
```

CREATE THE UNIT FILE at `/etc/systemd/system/pixelwise.service` on prod, matching the example from the theory section. Then reload systemd, enable the unit so it survives reboots, and start it:

```
sudo nano /etc/systemd/system/pixelwise.service
sudo systemctl daemon-reload
sudo systemctl enable pixelwise
sudo systemctl start pixelwise
sudo systemctl status pixelwise
```

KILL IT ON PURPOSE. A service that has never crashed is a service whose recovery has never been tested.

```
sudo systemctl stop pixelwise
sudo systemctl status pixelwise
sudo systemctl start pixelwise
journalctl -u pixelwise --since "5 min ago"
```

A NOTE ON WHAT COMES NEXT. The service runs, the contract is documented, the supervisor restarts crashes; what is missing is memory. GET `/results` returns an empty list because nothing is being stored. The next block adds PostgreSQL, wires POST `/classify` to write a row for every prediction, and replaces the stub with a real query against the database.

READ THE STATUS OUTPUT. Active: active (running) is what you want. Active: failed means the unit started and exited; the last few log lines below the status block usually say why. Active: inactive (dead) means the unit was never started.

FIRST END-TO-END LOOP ON THE NETWORK. From dev:

```
curl -X POST
http://192.168.56.11:8000/classify
-H "X-API-Key: ..." -d '{...}'
```

A live prediction from the server, served by systemd-supervised uvicorn, returns to your terminal on dev. The system is reachable across machines for the first time.

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thoughts during and after the exercises:

- What is the difference between running `uvicorn` manually and running it under `systemd`?
- Why does Pydantic validation matter for security as well as correctness?
- What happens when the `systemd` service crashes, and how does `Restart=always` help?
- Why is `GET /results` a stub right now, and what arrives in Block 6 to fill it in?
- How would you debug a failing endpoint using only `journalctl`?
- Which of the three rejection points (Pydantic, API key, rate limiter) is hardest to bypass, and why?

RECAP of key concepts:

- HTTP is a stateless request-response protocol; REST layers conventions on top so APIs are predictable across teams.
- Pydantic models are the API contract; FastAPI enforces them at the boundary and renders them as live documentation.
- `POST /classify`, `GET /results`, `GET /health` cover the surface this block exposes; `/results` is a deliberate stub for Block 6.
- API keys answer “who is calling”; rate limits answer “how often”.
- `systemd` turns a script into a service: starts on boot, restarts on crash, logs to `journalctl`.

MILESTONE. PixelWise runs as a managed `systemd` service on the prod VM. `POST /classify` returns live digit predictions; `GET /results` is stubbed pending persistence; `GET /health` reports liveness. You have killed and restarted the backend at least once and read the logs with `journalctl`. The next chapter wires the API to PostgreSQL so every prediction becomes a row that survives the response.

SECURITY THREAD. The API key lives in `.env`, never in code. Pydantic models are the input validation boundary; rate limits cap any single caller; the `X-API-Key` header keeps the secret out of access logs. HTTPS arrives in Block 7 with Nginx.

TEASER. Where do all those predictions go once the response has been sent?